TACITVS

Protocol Security Audit Report

Sine ira et studio.
"WITHOUT ANGER OR BIAS."

Tacitus, Annals 1.1

TACITVS

Auctor Princeps

August 8, 2025

Contents

1 Table of Contents **2**

2 Protocol Summary **2**

3 Disclaimer **2**

4 Risk Classification **2**

5 Audit Details **3**

5.1 The findings described in this document correspond the following commit hash:** 3

5.2 Scope 3

5.3 Roles 3

6 Executive Summary **3**

6.1 Issues found 3

7 Findings **3**

7.1 High 3

7.2 Informational 5

Proof & Connect **5**

Prepared by: TACITVS Lead Security Researcher: - I.Leskov(TACITVS)

I. Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - The findings described in this document correspond the following commit hash:**
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
  - Informational

II. Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a users passwords. The protocol is designed to be used by a single user, and not designed to be used by multiple users. Only the owner should be able to set and access this password.

III. Disclaimer

The TACITVS team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

IV. Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

V. Audit Details

A. The findings described in this document correspond the following commit hash:**

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

B. Scope

```
./src/
|-- PasswordStore.sol
```

C. Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set ot read password.

VI. Executive Summary

Add some notes about how the audit went, types of things you found, etc.

We spent X hours with Z auditors using Y tools, etc

A. Issues found

Secerity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

VII. Findings

A. High

[H-1] Password Storage on Public Blockchain
All data stored on-chain is visible to anyone. The `PasswordStore::s_password` variable can be read directly from blockchain storage, completely breaking the protocol's privacy.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool We use `1` because thats the storage slot of `s_password` in the contract

```
cast storage <CONTRACT_ADDRESS_HERE> 1
```

Youll get an output that looks like this:

`0x6d7950617373776f72640000000000000000000000000000000000000000000014`

You then parse that hex to string with:

```
cast parse-bytes32-string 0
   x6d7950617373776f72640000000000000000000000000000000000000000000014
```

And get an output of

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then sore the encrypted password on-chain. This would require user to remember another passwoord off-chain to decrypt the password. However, youd also likely want to remove view function as you wouldnt want the user to accidentally send a transaction with the password that decrypt your password.

Likelihood & Impact: - Impact: High - Likelihood: HIGH - Severity: HIGH

[H-2] Missing Access Control on setPassword()

The `PasswordStore::setpassword` function lacks `onlyOwner` modifier, allowing anyone to change the password and break the contract's intended functionality.

```
    function setPassword(string memory newPassword) external {
-->     // @audit - There are no access controls
        s_password = newPassword;
        emit SetNetPassword();
    }
```

Impact: Anyone can set/change the password of the contract, severly breaking the contract intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

Code

```
    function test_anyone_can_set_password(address randomAddress) public
        {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory expectedPassword = "myNewPassword";
        passwordStore.setPassword(expectedPassword);

        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(actualPassword, expectedPassword);
    }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function.

```
    if(msg.sender != s_owner){
        revert PasswordStore__NotOwner();
    }
```

Likelihood & Impact: - Impact: High - Likelihood: HIGH - Severity: HIGH

B. Informational

[I-1] Incorrect NatSpec Documentation
The `PasswordStore::getPassword` function documentation incorrectly specifies a parameter that doesn't exist in the function signature.

Description:

```
    /*
     * @notice This allows only the owner to retrieve the password.
-->  * @param newPassword The new password to set.
     */
    function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword` function signature is `getPassword()` which the natspec say it should be `getPassword(string)`

Impact: THe natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
-  * @param newPassword The new password to set.
```

Likelihood & Impact: - Impact: NONE - Likelihood: HIGH? - Severity: Informational/Gas/Non-Crits

Proof & Connect

This report is cryptographically anchored on-chain. Scan the left QR to verify the exact SHA-256 fingerprint and blockchain transaction. Scan the right QR to explore my portfolio or book a call.

Scan to verify on-chain proof

Scan for portfolio & booking